

Phase 1: Essential NFT Features - Implementation Documentation

Overview

This document provides comprehensive documentation for Phase 1 of the AuthiChain NFT Marketplace implementation. Phase 1 introduces essential NFT marketplace features including minting, collections, auctions, offers, and advanced search capabilities.

Implementation Date: October 18, 2025
Status:  Backend Complete |  Frontend In Progress
Version: 1.0.0

Table of Contents

- 1. [Database Schema](#)
 - 2. [IPFS Integration](#)
 - 3. [API Endpoints](#)
 - 4. [Features Overview](#)
 - 5. [Setup Instructions](#)
 - 6. [Usage Guide](#)
 - 7. [Testing](#)
 - 8. [Next Steps](#)
-

Database Schema

New Models Added

1. Collection

Represents a collection of NFTs (e.g., “Bored Apes”, “CryptoPunks”)

```

model Collection {
  id          String          @id @default(cuid())
  name        String
  slug         String          @unique
  description  String?
  coverImage   String?
  bannerImage  String?
  creatorId    String
  category     CollectionCategory @default(ART)
  floorPrice   Float?
  totalVolume  Float          @default(0)
  itemCount    Int            @default(0)
  ownerCount   Int            @default(0)
  isVerified   Boolean        @default(false)
  royaltyPercentage Float    @default(0)
  blockchain   String          @default("ethereum")
  contractAddress String?
  isPublic     Boolean        @default(true)
  tags         String[]
  socialLinks  Json?
  metadata     Json?
  createdAt    DateTime       @default(now())
  updatedAt    DateTime       @updatedAt
  nfts         NFT[]
}

```

Categories:

- ART
- PHOTOGRAPHY
- MUSIC
- VIDEO
- GAMING
- SPORTS
- COLLECTIBLES
- UTILITY
- DOMAIN_NAMES
- VIRTUAL_WORLDS
- TRADING_CARDS
- MEMBERSHIP
- OTHER

2. NFT

Main NFT model with comprehensive metadata and IPFS support

```

model NFT {
  id                String           @id @default(cuid())
  tokenId           String           @unique
  contractAddress   String
  blockchain        String           @default("ethereum")
  name              String
  description        String?
  image             String
  imageIpfs         String?
  animationUrl      String?
  externalUrl       String?
  metadata          Json?
  metadataIpfs      String?
  collectionId      String?
  creatorId         String
  ownerId           String
  price             Float?
  currency          String           @default("ETH")
  royaltyPercentage Float           @default(0)
  status            NftStatus        @default(ACTIVE)
  listingType       ListingType      @default(FIXED_PRICE)
  isVerified        Boolean          @default(false)
  authenticityScore Int?
  attributes        Json?
  rarityScore       Float?
  rarityRank        Int?
  viewCount         Int              @default(0)
  likeCount         Int              @default(0)
  isMinted          Boolean          @default(false)
  mintTransactionHash String?
  mintedAt          DateTime?
  lastSalePrice     Float?
  lastSaleDate      DateTime?
  createdAt         DateTime         @default(now())
  updatedAt         DateTime         @updatedAt
}

```

Status Values:

- ACTIVE - NFT is active and available
- INACTIVE - NFT is temporarily inactive
- DELETED - NFT is deleted
- LISTED - NFT is listed for sale
- UNLISTED - NFT is not for sale
- IN_AUCTION - NFT is in an active auction
- SOLD - NFT has been sold
- BURNED - NFT has been burned

Listing Types:

- FIXED_PRICE - Listed at fixed price
- AUCTION - Available in auction
- NOT_FOR_SALE - Not currently for sale

3. Auction

Auction system for NFTs with bidding functionality

```

model Auction {
  id String @id @default(cuid())
  nftId String
  sellerId String
  startPrice Float
  reservePrice Float?
  currentBid Float?
  currentBidderId String?
  bidIncrement Float @default(0.01)
  startTime DateTime @default(now())
  endTime DateTime
  status AuctionStatus @default(ACTIVE)
  winnerId String?
  winningBid Float?
  settledAt DateTime?
  transactionHash String?
  bidCount Int @default(0)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  nft NFT @relation(fields: [nftId], references: [id])
  bids Bid[]
}

```

Auction Status:

- PENDING - Auction scheduled to start
- ACTIVE - Auction is ongoing
- ENDED - Auction has ended
- SETTLED - Auction completed and NFT transferred
- CANCELLED - Auction was cancelled

4. Bid

Individual bids placed on auctions

```

model Bid {
  id String @id @default(cuid())
  auctionId String?
  nftId String?
  bidderId String
  amount Float
  status BidStatus @default(ACTIVE)
  transactionHash String?
  expiresAt DateTime?
  createdAt DateTime @default(now())
}

```

Bid Status:

- ACTIVE - Bid is active
- OUTBID - Bid has been outbid
- WINNING - Currently winning bid
- WON - Winning bid of completed auction
- LOST - Lost the auction
- CANCELLED - Bid was cancelled
- EXPIRED - Bid has expired

5. Offer

Offer system for direct NFT purchases

```
model Offer {
  id          String      @id @default(cuid())
  nftId       String
  offererId   String
  amount      Float
  currency    String      @default("ETH")
  message     String?
  status      OfferStatus @default(PENDING)
  expiresAt   DateTime
  acceptedAt  DateTime?
  rejectedAt  DateTime?
  cancelledAt  DateTime?
  transactionHash String?
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
}
```

Offer Status:

- PENDING - Offer is pending review
- ACCEPTED - Offer has been accepted
- REJECTED - Offer has been rejected
- CANCELLED - Offer was cancelled by offerer
- EXPIRED - Offer has expired

6. NFTTransfer

Transaction history for NFT transfers

```
model NFTTransfer {
  id          String      @id @default(cuid())
  nftId       String
  fromAddress String
  toAddress   String
  transactionHash String
  blockchain  String      @default("ethereum")
  transferType TransferType @default(SALE)
  price       Float?
  currency    String?
  timestamp   DateTime     @default(now())
}
```

Transfer Types:

- MINT - Initial minting
- SALE - Sale transaction
- TRANSFER - Regular transfer
- BURN - NFT burned

7. PriceHistory

Track price changes and sales over time

```
model PriceHistory {
  id          String @id @default(cuid())
  nftId       String
  price       Float
  currency    String @default("ETH")
  eventType   PriceEventType @default(LISTING)
  timestamp   DateTime @default(now())
}
```

Event Types:

- LISTING - NFT listed for sale
- SALE - NFT sold
- DELISTING - NFT removed from sale
- PRICE_CHANGE - Price updated

IPFS Integration

Configuration

IPFS integration uses **NFT.Storage** for decentralized file storage.

Setup:

1. Get API key from [NFT.Storage](https://nft.storage/) (<https://nft.storage/>)
2. Add to `.env` :

```
env
```

```
NFT_STORAGE_API_KEY=your_api_key_here
```

Available Functions

All IPFS functions are available in `/lib/ipfs.ts` :

1. Upload File to IPFS

```
uploadFileToIPFS(
  file: Buffer | Uint8Array,
  filename: string,
  contentType?: string
): Promise<IPFSUploadResult>
```

2. Upload Image from URL

```
uploadImageFromURL(imageUrl: string): Promise<IPFSUploadResult>
```

3. Upload NFT Metadata

```
uploadMetadataToIPFS(metadata: NFTMetadata): Promise<IPFSUploadResult>
```

4. Upload Complete NFT

```
uploadNFTToIPFS(
  imageFile: Buffer | Uint8Array,
  imageFilename: string,
  metadata: Omit<NFTMetadata, 'image'>,
  imageContentType?: string
): Promise<{
  imageResult: IPFSUploadResult;
  metadataResult: IPFSUploadResult;
}>
```

5. Batch Upload

```
uploadBatchToIPFS(
  files: Array<{
    file: Buffer | Uint8Array;
    filename: string;
    metadata: Omit<NFTMetadata, 'image'>;
    contentType?: string;
  }>
): Promise<Array<{
  imageResult: IPFSUploadResult;
  metadataResult: IPFSUploadResult;
}>>
```

Utility Functions

```
getGatewayUrl(ipfsUrl: string): string
getCIDFromUrl(ipfsUrl: string): string
isIPFSConfigured(): boolean
getIPFSStatus(): Promise<{ configured: boolean; working: boolean; error?: string }>
```

API Endpoints

NFT Endpoints

Mint NFT

```
POST /api/nft/mint
```

Request Body:

```
{
  "mode": "single" || "batch",
  "nft": { // For single mode
    "name": "My NFT",
    "description": "Description",
    "image": "https://static01.nyt.com/images/2021/03/11/arts/11nft-explain-1/merlin_184196631_939fb22d-b909-4205-99d9-b464fb961d32-superJumbo.jpg",
    "imageFile": "base64...", // Optional, for IPFS upload
    "animationUrl": "https://queue-it.com/media/ss1dxknh/bored-apes.jpg",
    "externalUrl": "https://compote.slate.com/images/bbc1b467-8450-440b-a9b2-4113f13c24ea.jpeg?crop=1560%2C1040%2Cx0%2Cy0",
    "collectionId": "collection_id",
    "attributes": [
      {
        "trait_type": "Background",
        "value": "Blue"
      }
    ],
    "royaltyPercentage": 10,
    "price": 1.5,
    "currency": "ETH",
    "blockchain": "ethereum"
  },
  "nfts": [] // For batch mode
}
```

Response:

```
{
  "success": true,
  "nft": { /* NFT object */ },
  "message": "NFT minted successfully"
}
```

List NFTs

```
GET /api/nft/list?page=1&limit=20&status=ACTIVE&sortBy=createdAt&sortOrder=desc
```

Query Parameters:

- page - Page number (default: 1)
- limit - Items per page (default: 20)
- collectionId - Filter by collection
- creatorId - Filter by creator
- ownerId - Filter by owner
- status - Filter by status
- listingType - Filter by listing type
- minPrice - Minimum price
- maxPrice - Maximum price
- verified - Filter verified NFTs
- category - Filter by collection category
- sortBy - Sort field
- sortOrder - Sort direction (asc/desc)

Get NFT Details

```
GET /api/nft/[id]
```

Response:

```
{
  "success": true,
  "nft": {
    /* Full NFT details including:
      - Collection
      - Active auctions
      - Pending offers
      - Transfer history
      - Price history
    */
  }
}
```

Update NFT

```
PUT /api/nft/[id]
```

Request Body:

```
{
  "name": "Updated Name",
  "description": "Updated description",
  "price": 2.0,
  "status": "LISTED",
  "listingType": "FIXED_PRICE"
}
```

Search NFTs

```
GET /api/nft/search?q=keyword&page=1&limit=20
```

Query Parameters:

- `q` - Search query (required)
- `page` - Page number
- `limit` - Items per page
- `collectionId` - Filter by collection
- `category` - Filter by category
- `minPrice` - Minimum price
- `maxPrice` - Maximum price
- `status` - Filter by status
- `sortBy` - Sort field
- `sortOrder` - Sort direction

Collection Endpoints

Create Collection

POST /api/collections/create

Request Body:

```
{
  "name": "My Collection",
  "description": "Collection description",
  "coverImage": "https://i.etsystatic.com/39972408/r/il/b4ad04/6474265233/il_570xN.6474265233_ftfi.jpg",
  "bannerImage": "https://upload.wikimedia.org/wikipedia/commons/d/d3/Edward_Penfield%2C_Harper%27s_June%2C_1896.jpg",
  "category": "ART",
  "royaltyPercentage": 5,
  "blockchain": "ethereum",
  "contractAddress": "0x...",
  "tags": ["art", "digital"],
  "socialLinks": {
    "twitter": "https://twitter.com/...",
    "discord": "https://discord.gg/...",
    "website": "https://..."
  }
}
```

List Collections

GET /api/collections/list?page=1&limit=20

Query Parameters:

- page - Page number
- limit - Items per page
- category - Filter by category
- creatorId - Filter by creator
- verified - Filter verified collections
- sortBy - Sort field
- sortOrder - Sort direction

Get Collection Details

GET /api/collections/[slug]

Update Collection

PUT /api/collections/[slug]

Delete Collection

DELETE /api/collections/[slug]

Auction Endpoints

Create Auction

```
POST /api/auctions/create
```

Request Body:

```
{
  "nftId": "nft_id",
  "startPrice": 1.0,
  "reservePrice": 2.0,
  "bidIncrement": 0.1,
  "durationHours": 24,
  "startTime": "2025-10-20T00:00:00Z" // Optional
}
```

Place Bid

```
POST /api/auctions/bid
```

Request Body:

```
{
  "auctionId": "auction_id",
  "amount": 1.5
}
```

List Auctions

```
GET /api/auctions/list?page=1&limit=20&status=ACTIVE
```

Query Parameters:

- page - Page number
- limit - Items per page
- status - Filter by status
- nftId - Filter by NFT
- sellerId - Filter by seller
- sortBy - Sort field
- sortOrder - Sort direction

Get Auction Details

```
GET /api/auctions/[id]
```

End/Cancel Auction

```
POST /api/auctions/[id]
```

Request Body:

```
{
  "action": "end" || "cancel"
}
```

Offer Endpoints

Create Offer

POST /api/offers/create

Request Body:

```
{
  "nftId": "nft_id",
  "amount": 1.5,
  "currency": "ETH",
  "message": "Optional message",
  "expirationDays": 7
}
```

List Offers

GET /api/offers/list?type=received&status=PENDING

Query Parameters:

- type - "received" or "made"
- status - Filter by status
- nftId - Filter by NFT
- page - Page number
- limit - Items per page

Accept Offer

POST /api/offers/[id]/accept

Reject Offer

POST /api/offers/[id]/reject

Cancel Offer

POST /api/offers/[id]/cancel

Features Overview

1. NFT Minting System

Capabilities:

- Single NFT minting
- Batch minting (multiple NFTs at once)
- IPFS storage for images and metadata
- Customizable metadata and attributes
- Royalty configuration
- Collection assignment
- Multiple blockchain support

Metadata Support:

- Name and description
- Images (IPFS or URL)
- Animation URLs (for videos/audio)
- External URLs
- Custom attributes/traits
- Rarity scores

2. Collection Management

Features:

- Create and manage collections
- Collection categories
- Cover and banner images
- Royalty settings
- Verification badges
- Social media links
- Collection statistics (floor price, volume, item count)
- Public/private collections

3. Auction System

Capabilities:

- Timed auctions
- Reserve prices
- Bid increments
- Real-time bidding
- Automatic auction ending
- Winner determination
- Bid history
- Auction cancellation

Bid Management:

- Minimum bid requirements
- Outbid notifications
- Bid status tracking
- Winning bid settlement

4. Offer/Counter-Offer System

Features:

- Make offers on any NFT
- Offer expiration
- Accept/reject offers
- Cancel offers
- Offer messages
- Multiple offer management
- Automatic offer rejection on sale

5. Advanced Search & Filtering

Search Capabilities:

- Text search (name, description)
- Category filtering
- Price range filtering
- Status filtering
- Collection filtering
- Verified NFTs only
- Multiple sort options

Available Filters:

- Price (min/max)
- Status (active, listed, in auction, etc.)
- Listing type (fixed price, auction, not for sale)
- Category
- Verified status
- Collection
- Creator
- Owner

6. NFT Detail Pages

Information Displayed:

- Full metadata
- Ownership history
- Transfer history
- Price history (chart-ready data)
- Active auctions
- Pending offers
- Authenticity score
- Verification status
- Collection info
- View count

Available Actions:

- Buy now (fixed price)
- Make offer
- Place bid (if in auction)
- Update listing (if owner)
- View on blockchain
- Share

7. Marketplace Pages

Explore Page:

- All NFTs
- Advanced filtering
- Grid/list views
- Sort options

Collections Page:

- All collections
- Category filtering
- Featured collections
- Verified collections

Auctions Page:

- Active auctions
 - Ending soon
 - Recently ended
 - Sort by bid count, end time, etc.
-

Setup Instructions

Prerequisites

1. Node.js 18+ installed
2. PostgreSQL database
3. NFT.Storage API key

Installation Steps

1. Install Dependencies:

```
bash
cd /home/ubuntu/authichain_premium/app
npm install
```

2. Configure Environment:

Add to `.env` file:

```
env
NFT_STORAGE_API_KEY=your_nft_storage_api_key
DATABASE_URL=your_database_url
```

3. Update Database Schema:

```
bash
npx prisma generate
npx prisma db push
```

4. Verify Installation:

```
bash
npm run dev
```

Testing IPFS Integration

```
# Create test script
node -e "
const { getIPFSStatus } = require('./lib/ipfs');
getIPFSStatus().then(status => {
  console.log('IPFS Status:', status);
});
"
```

Usage Guide

For Creators

Minting an NFT

1. **Prepare Your NFT:**

- Create artwork (image, video, etc.)
- Prepare metadata (name, description, attributes)
- Decide on royalties and pricing

2. **Create Collection (Optional):**

```
bash
```

```
POST /api/collections/create
```

3. **Mint NFT:**

```
bash
```

```
POST /api/nft/mint
```

4. **List for Sale:**

- Set fixed price, or
- Create auction

Creating an Auction

1. **Prepare NFT:**

- Must own the NFT
- NFT must not be in another auction

2. **Create Auction:**

```
bash
```

```
POST /api/auctions/create
```

3. **Monitor Bids:**

- View bid history
- Check current highest bid
- Watch countdown

4. **End Auction:**

- Automatically ends at end time
- Manually end with POST /api/auctions/[id]

For Collectors

Buying an NFT

Fixed Price:

1. Browse marketplace
2. View NFT details
3. Click “Buy Now”
4. Complete transaction

Auction:

1. Find active auction
2. Place bid (must exceed current bid + increment)
3. Monitor auction
4. Wait for auction end

Make Offer:

1. View NFT
2. Click “Make Offer”
3. Set amount and expiration
4. Wait for acceptance

Managing Offers

Made Offers:

- View in “My Offers” dashboard
- Cancel pending offers
- See accepted/rejected offers

Received Offers:

- View offers on your NFTs
- Accept or reject
- Send counter-offer (via message)

Testing

API Testing

Use the included test scripts or tools like Postman:

Test NFT Minting

```
curl -X POST http://localhost:3000/api/nft/mint \
-H "Content-Type: application/json" \
-H "Cookie: next-auth.session-token=YOUR_SESSION" \
-d '{
  "mode": "single",
  "nft": {
    "name": "Test NFT",
    "description": "Test Description",
    "image": "https://www.datocms-assets.com/16499/1622711604-nfts21examples.png?
auto=format",
    "price": 1.0
  }
}'
```

Test Collection Creation

```
curl -X POST http://localhost:3000/api/collections/create \
-H "Content-Type: application/json" \
-H "Cookie: next-auth.session-token=YOUR_SESSION" \
-d '{
  "name": "Test Collection",
  "category": "ART"
}'
```

Test Search

```
curl "http://localhost:3000/api/nft/search?q=test&page=1&limit=10"
```

Database Testing

```
# Check if tables exist
npx prisma studio
```

Browse to <http://localhost:5555> to view data.

Next Steps

Phase 2 Recommendations

1. Frontend Implementation:

- Minting UI components
- Collection management pages
- Auction interface with countdown
- Offer management dashboard
- NFT detail pages
- Marketplace browse pages

2. Smart Contract Integration:

- ERC-721 contract deployment
- Minting to blockchain

- Transfer functionality
- Royalty enforcement

3. **Advanced Features:**

- Lazy minting
- Gasless transactions
- Multi-chain support
- Advanced analytics
- Notification system

4. **Payment Integration:**

- Crypto wallet payments
- Fiat on-ramp
- Escrow system
- Royalty distribution

5. **Security Enhancements:**

- Image verification
- Copyright checking
- Fraud detection
- Rate limiting

Support & Documentation

API Reference

All endpoints are documented above with request/response examples.

Database Schema

Full schema available in `/prisma/schema.prisma`

IPFS Functions

Documentation in `/lib/ipfs.ts`

Issues & Questions

Contact development team or review code comments for detailed implementation notes.

Version History

v1.0.0 - October 18, 2025

- Initial Phase 1 implementation
- Database schema updates
- IPFS integration
- Complete API backend
- Minting system
- Collection management
- Auction system

- Offer system
- Search and filtering

License

Copyright © 2025 AuthiChain. All rights reserved.